

Lecture 5

Sorting

15-122: Principles of Imperative Computation (Spring 2025)
Frank Pfenning, Rob Simmons

We begin this lecture by discussing how to compare running times of functions in an abstract, mathematical way. The same underlying mathematics can be used for other purposes, like comparing memory consumption or the amount of parallelism permitted by an algorithm. We then use this to take a first look at sorting algorithms, of which there are many. In this lecture it will be selection sort because of its simplicity.

Additional Resources

- [Review slides](https://cs.cmu.edu/~15122/handouts/slides/review/05-bigo.pdf) (<https://cs.cmu.edu/~15122/handouts/slides/review/05-bigo.pdf>)
- [Code for this lecture](https://cs.cmu.edu/~15122/handouts/code/05-bigo.tgz) (<https://cs.cmu.edu/~15122/handouts/code/05-bigo.tgz>)

In terms of our learning goals, we will work on:

Computational Thinking: Still trying to understand how order can lead to efficient computation. Worst-case asymptotic complexity of functions.

Algorithms and Data Structures: Sorting of arrays in general, and selection sort in particular. Big-O notation.

Programming: More examples of programming with arrays and algorithm invariants.

1 Big-O Notation

In the design and analysis of algorithms, we try to make the running time of functions mathematically precise by deriving so-called *asymptotic complexity measures* for algorithms. In addition for wanting mathematical precision, there are two fundamental principles that guide our mathematical analysis.

1. We want an analysis that is *practically useful*. This has two consequences.

First, we observe that the problems we care about are ones that get harder as our inputs get bigger, so our definition of Big- O captures the idea that we only care about the behavior of an algorithm *on large inputs*, that is, when it takes a long time. It is when the inputs are large that differences between algorithms become really pronounced.

Second, there is another mathematical concept, Big- Θ , which you can read about on your own and which is frequently the concept that we actually want to talk about in this class. But computer scientists definitely tend to think and talk and communicate in terms of Big- O notation. We teach Big- O in part to help you communicate with other computer scientists!

2. We want an analysis that is *enduring*. One consequence of this is that we want our analysis to be the same even given computers that work very different than the ones we use — in particular, ones that are much faster than the ones we use.

The only way to handle this is to say that we don't care about *constant factors* in the mathematical analysis of how long it takes our program to run. In practice, constant factors can make a big difference, but they are influenced by so many factors (compiler, runtime system, machine model, available memory, etc.) that at the abstract, mathematical level a precise analysis is neither appropriate nor feasible.

Let's see how these two fundamental principles guide us in the comparison between functions that measure the running time of an algorithm.

Let's say we have functions f and g that measure the number of operations of an algorithm as a function of the size of the input. For example $f(n) = 3 \log n$ measures the number of comparisons performed in binary search for an array of size n , and $g(n) = 3n$ measures the number of comparisons performed in linear search for an array of size n .

The simplest form of comparison would be

$$f \leq_0 g \text{ if for every } n \geq 0, f(n) \leq g(n).$$

However, this violates principle (1) because we compare the values and f and g on all possible inputs n .

We can refine this by saying that *eventually*, f will always be smaller than or equal to g . We express "eventually" by requiring that there be a number n_0 such that $f(n) \leq g(n)$ for all n that are greater than n_0 .

$$f \leq_1 g \text{ if there is some natural number } n_0 \text{ such that for every } n \geq n_0 \\ \text{it is the case that } f(n) \leq g(n).$$

This now incorporates the first principle (we only care about the function on large inputs), but constant factors still matter. For example, according to the last definition we have $3n \leq_1 5n$ but $5n \not\leq_1 3n$. But if constant factors don't matter, then the two should be equivalent. We can repair this by allowing the right-hand side to be multiplied by a suitable positive real number.

$f \leq_2 g$ if there is a real constant $c > 0$ and some natural number n_0 such that for every $n \geq n_0$ we have $f(n) \leq c \times g(n)$.

This definition is now appropriate.

The less-or-equal symbol $\leq$ is already overloaded with many meanings, so we write instead:

$f \in O(g)$ if there is a real constant $c > 0$ and some natural number n_0 such that for every $n \geq n_0$ we have $f(n) \leq c \times g(n)$.

This notation derives from the view of $O(g)$ as a set of functions, namely those that eventually are smaller than a constant times g .¹ Just to be explicit, we also write out the definition of $O(g)$ as a set of functions:

$$O(g) = \{f \mid \text{there are } c > 0 \text{ and } n_0 \text{ s.t. for all } n \geq n_0, f(n) \leq c \times g(n)\}$$

With this definition we can check that $O(g(n)) = O(c \times g(n))$.

When we characterize the running time of a function using big-O notation we refer to it as the *asymptotic complexity* of the function. Here, *asymptotic* refers to the fundamental principles listed above: we only care about the function in the long run, and we ignore constant factors. Usually, we use an analysis of the *worst case* among the inputs of a given size. Trying to do *average case* analysis is much harder, because it depends on the distribution of inputs. Since we often don't know the distribution of inputs it is much less clear whether an average case analysis may apply in a particular use of an algorithm.

The asymptotic worst-case time complexity of linear search is $O(n)$, which we also refer to as *linear time*. The worst-case asymptotic time complexity of binary search is $O(\log n)$, which we also refer to as *logarithmic time*. *Constant time* is usually described as $O(1)$, expressing that the running time is independent of the size of the input.

Some brief fundamental facts about big-O. For any polynomial, only the highest power of n matters, because it eventually comes to dominate the function. For example, $O(5n^2 + 3n + 83) = O(n^2)$. Also $O(\log n) \subseteq O(n)$, but $O(n) \not\subseteq O(\log n)$.

¹In textbooks and research papers you may sometimes see this written as $f = O(g)$ but that is questionable, comparing a function with a set of functions.

That is the same as to say $O(\log n) \subsetneq O(n)$, which means that $O(\log n)$ is a proper subset of $O(n)$, that is, $O(\log n)$ is a subset ($O(\log n) \subseteq O(n)$), but they are not equal ($O(\log n) \neq O(n)$). Logarithms to different (constant) bases are asymptotically the same: $O(\log_2 n) = O(\log_b n)$ because $\log_b n = \log_2 n / \log_2 b$.

As a side note, it is mathematically correct to say the worst-case running time of binary search is $O(n)$, because $\log n \in O(n)$. It is, however, a looser characterization than saying that the running time of binary search is $O(\log n)$, which is also correct. Of course, it would be incorrect to say that the running time is $O(1)$. Generally, when we ask you to characterize the worst-case running time of an algorithm we are asking for the tightest bound in big-O notation.

There is nothing special about the variable n . We can use a different variable, such as x , to say $4x + \log_9 x + 2 \in O(x)$, and we can generalize to multiple variables to say that $2w + 2h^2 + 4 \in O(w + h^2)$. To formalize this, we say that there is a single constant c , but we pick a different starting point w_0 and h_0 for every variable.

2 Sorting Algorithms

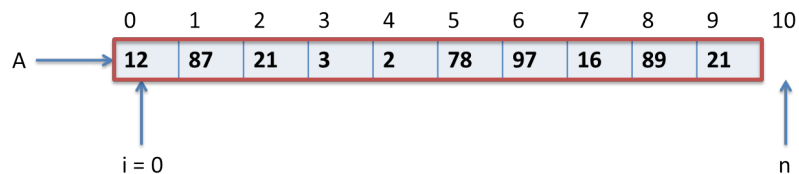
We have seen in the last lecture that having a sorted array can make it easier to do search. This suggests that it may be important to be able to take an unsorted array and rearrange it so it's sorted!

There are many different algorithms for sorting: bucket sort, bubble sort, insertion sort, selection sort, heap sort, etc. This is testimony to the importance and complexity of the problem, despite its apparent simplicity. In this lecture we discuss selection sort, which is one of the simplest algorithms.

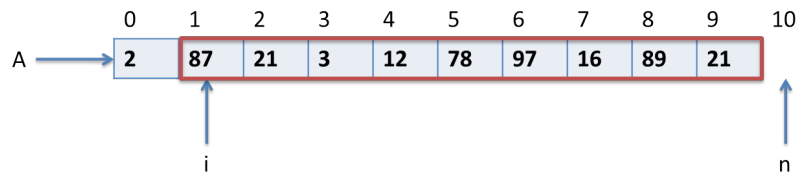
3 Selection Sort

Selection sort is based on the idea that on each iteration we select the *smallest* element of the part of the array that has not yet been sorted and move it to the end of the sorted part at the beginning of the array.

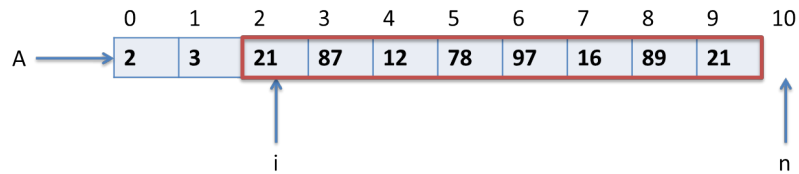
Let's play this through for two steps on an example array. Initially, we consider the whole array (from $i = 0$ to the end). We write this as $A[0..n)$, that is the segment of the array starting at 0 up to n , where n is *excluded*.



We now find the minimal element of the array segment under consideration (2) and move it to the front of the array. What do we do with the element that is there? We move it to the place where 2 was (namely at $A[4]$). In other words, we *swap* the first element with the minimal element. Swapping is a useful operation when sorting an array passed as input modifying it, because the result of a correct sort must be a permutation of the input. If swapping is our *only* operation we are immediately guaranteed that the result is a permutation of the input.



Now 2 is in the right place, and we find the smallest element in the remaining array segment and move it to the beginning of the segment ($i = 1$).



Let's pause and see if we can write down properties of the variables and array segments that allow us to write the code correctly. First we observe rather straightforwardly that

$$0 \leq i \leq n$$

where $i = n$ after the last iteration and $i = 0$ before the first iteration. Next we observe that the elements to the left of i are already sorted.

$$A[0..i) \text{ sorted}$$

These two invariants are true initially and suffice to imply the post-condition. However, it won't be possible to prove the correctness of selection sort because we can't prove that these two invariants, on their own, are preserved by every iteration of the loop. We also need to know that *all* elements to the left of i are less or equal to *all* element to the right of i . We abbreviate this:

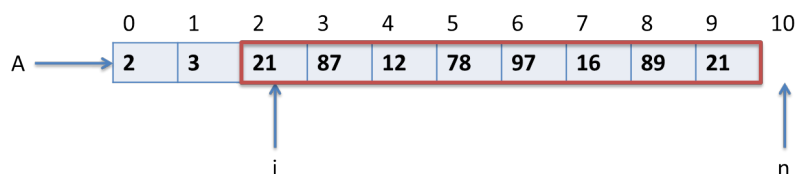
$$A[0..i) \leq A[i..n)$$

saying that every element in the left segment is smaller than or equal to every element in the right segment.

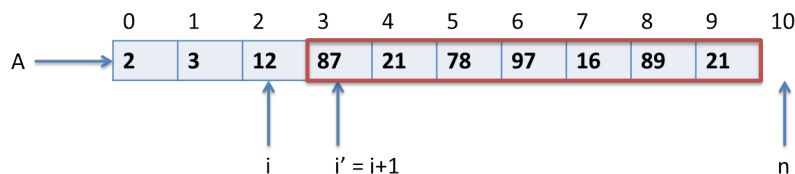
We summarize the invariants

$$\begin{aligned} 0 \leq i \leq n \\ A[0..i) \text{ sorted} \\ A[0..i) \leq A[i..n) \end{aligned}$$

Let's reason through *without any code* (for the moment), why these invariants are preserved. Let's look at the picture again.



In the next iteration we pick the minimal element among $A[i..n)$, which would be $12 = A[4]$. We now swap this to $i = 2$ and increment i . We write here $i' = i + 1$ in order to distinguish the old value of i from the new one, as we do in proofs of preservation of the loop invariant.



Since we only step when $i < n$, the bounds on i are preserved.

Why is $A[0..i+1)$ sorted? We know by the third invariant that any element in $A[0..i)$ is less than or equal to any element in $A[i..n)$ and in particular the one we moved to $A[i+1]$. And $A[0..i)$ was already sorted before by the second invariant.

Why is $A[0..i+1) \leq A[i+1..n)$? We know from the loop invariant before the iteration that $A[0..i) \leq A[i+1..n)$. So it remains to show that $A[i..i+1) \leq A[i+1..n)$. But that is true since $A[i]$ was a minimal element of $A[i..n)$ which is the same as saying that it is smaller or equal to all the elements in $A[i..n)$ and therefore also $A[i+1..n)$ after we swap the old $A[i]$ into its new position.

4 Programming Selection Sort

From the above invariants and description of the algorithm, the correct code is simple to write, including its invariants. The function does not return a value, since it modifies the given array A , so it has declaration:

```
void sort(int[] A, int lo, int hi)
//@requires 0 <= lo && lo <= hi && hi <= \length(A);
//@ensures is_sorted(A, lo, hi);
;
```

We encourage you to now write the function, using the following auxiliary and contract functions:

1. `is_sorted(A, lo, hi)` which is true if the array segment $A[lo..hi]$ is sorted.
2. `le_seg(x, A, lo, hi)` which is true if $x \leq A[lo..hi]$ (which means that x is less than or equal to all elements in the array segment).
3. `le_segs(A, lo1, hi1, B, lo2, hi2)` which is true if $A[lo_1..hi_1] \leq B[lo_2..hi_2]$ (which means all elements in the first segment are less than or equal to the all elements in the second array segment).
4. `swap(A, i, j)` modifies the array A by swapping $A[i]$ with $A[j]$. Of course, if $i = j$, the array remains unchanged.
5. `find_min(A, lo, hi)` which returns the index m of a minimal element in the non-empty segment $A[lo..hi]$.

Please write it and then compare it to our version on the next page.


```
1 void sort(int[] A, int lo, int hi)
2 //@requires 0 <= lo && lo <= hi && hi <= \length(A);
3 //@ensures is_sorted(A, lo, hi);
4 {
5     for (int i = lo; i < hi; i++)
6         //@loop_invariant lo <= i && i <= hi;
7         //@loop_invariant is_sorted(A, lo, i);
8         //@loop_invariant le_segs(A, lo, i, A, i, hi);
9         {
10             int min = find_min(A, i, hi);
11             swap(A, i, min);
12         }
13 }
```

At this point, let us verify that the loop invariants are initially satisfied.

- $lo \leq i$ and $i \leq hi$ since $i = lo$ and $lo \leq hi$ (by the **@requires** precondition on line 2).
- $A[lo..i]$ is sorted, since for $i = lo$ the segment $A[lo..lo]$ is empty (has no elements) since the right bound is exclusive.
- $A[lo..i] \leq A[i..hi]$ is true since for $i = lo$ the segment $A[lo..lo]$ has no elements. The other segment, $A[lo..hi]$, is the whole part of the array that is supposed to be sorted.

How can we prove the post-condition (**@ensures**) of the sorting function? By the loop invariant $lo \leq i \leq hi$ and the negation of the loop condition $i \geq hi$ we know $i = hi$. The second loop invariant then states that $A[lo..hi]$ is sorted, which is the post-condition.

5 Auxiliary Functions

Besides the specification functions in contracts, we also used two auxiliary functions: `swap` and `find_min`.

Here is the implementation of `swap`.

```
1 void swap(int[] A, int i, int j)
2 //@requires 0 <= i && i < \length(A);
3 //@requires 0 <= j && j < \length(A);
4 {
5     int tmp = A[i];
6     A[i] = A[j];
7     A[j] = tmp;
8 }
```

For `find_min`, we recommend you follow the method used for selection sort: follow the algorithm for a couple of steps on a generic example, write down the invariants in general terms, and then synthesize the simple code and invariants from the result. What we have is below, for completeness.

```
1 int find_min(int[] A, int lo, int hi)
2 //@requires 0 <= lo && lo < hi && hi <= \length(A);
3 //@ensures lo <= \result && \result < hi;
4 //@ensures le_seg(A[\result], A, lo, hi);
5 {
6     int min = lo;
7     for (int i = lo+1; i < hi; i++)
8         //@loop_invariant lo < i && i <= hi;
9         //@loop_invariant lo <= min && min < hi;
10        //@loop_invariant le_seg(A[min], A, lo, i);
11        {
12            if (A[i] < A[min]) {
13                min = i;
14            }
15        }
16
17     return min;
18 }
```

6 Asymptotic Complexity Analysis

Previously, we have had to prove that functions actually terminate. Here we do a more detailed argument: we do counting in order to give a big-O

classification of the number of operations. If we have an explicit bound on the number of operations that, of course, implies termination.

Assume $lo = 0$ and $hi = n$ for notational simplicity. The loop in function `sort` iterates n times, from $i = 0$ to $i = n - 1$. Actually, we could stop one iteration earlier, but that does not effect the asymptotic complexity, since it only involves a constant number of additional operations.

For each iteration of this loop (identified by the value for i), `find_min` does a linear search through the array segment to the right of i . We then do a simple swap. The linear search will take $n - i - 1$ iterations, and cannot be easily improved since the array segment $A[i + 1..n]$ is not (yet) sorted. So the total number of iterations (counting the number of inner iterations for each outer one)

$$(n - 1) + (n - 2) + (n - 3) + \cdots + 0 = \frac{n(n - 1)}{2}$$

During each of these iterations, we only perform a constant amount of operations (some comparisons, assignments, and increments), so, asymptotically, the running time can be estimated as

$$O\left(\frac{n(n - 1)}{2}\right) = O\left(\frac{n^2}{2} - \frac{n}{2}\right) = O(n^2)$$

The last equation follows since for a polynomial, as we remarked earlier, only the degree matters.

We summarize this by saying that the worst-case running time of selection sort is quadratic. In this algorithm there isn't a significant difference between average case and worst case analysis: the number of iterations is exactly the same, and we only save one or two assignments per iteration in the loop body of the `find_min` function if the array is already sorted.

7 Empirical Validation

If the running time is really $O(n^2)$ and not asymptotically faster, we predict the following: for large inputs, its running time should be essentially cn^2 for some constant c . If we *double* the size of the input to $2n$, then the running time should roughly become $c(2n)^2 = 4(cn^2)$ which means the function should take approximately 4 times as many seconds as before.

We try this with the function `sort_time(n, r)` which generates a random array of size n and then sorts it r times. You can find the C0 code as `sort-time.c0` in this lecture's code directory. We run this code several times, with different parameters.

```
% cc0 selectsort.c0 sort-time.c0
% time ./a.out -n 1000 -r 100
Timing array of size 1000, 100 times
0
0.700u 0.001s 0:00.70 100.0%    0+0k 0+0io 0pf+0w
% time ./a.out -n 2000 -r 100
Timing array of size 2000, 100 times
0
2.700u 0.001s 0:02.70 100.0%    0+0k 0+0io 0pf+0w
% time ./a.out -n 4000 -r 100
Timing array of size 4000, 100 times
0
10.790u 0.002s 0:10.79 100.0%    0+0k 0+0io 0pf+0w
% time ./a.out -n 8000 -r 100
Timing array of size 8000, 100 times
0
42.796u 0.009s 0:42.80 99.9%    0+0k 0+0io 0pf+0w
%
```

Calculating the ratios of successive running times, we obtain

n	Time	Ratio
1000	0.700	
2000	2.700	3.85
4000	10.790	4.00
8000	42.796	3.97

We see that especially for the larger numbers, the ratio is almost exactly 4 when doubling the size of the input. Our conjecture of quadratic asymptotic running time has been experimentally confirmed.

Exercises

Exercise 1 (sample solution on page 16). Give the big-O bounds for the following two functions:

```
int function1(int[] A, int n)
//@requires n == \length(A);
{
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            count = count + A[j] * (i-1);
        }
    }
    return count;
}
```

and

```
int function2(int[] A, int n)
//@requires (n == \length(A));
{
    int count = n;
    int k = 5;
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < k; j++) {
            count = count + i * (j+1);
        }
    }
    return count;
}
```

Exercise 2 (sample solution on page 16). Give the simplest and tightest big-O bounds of the following functions:

- $f_1(x) = 3x^4 + \sqrt{x} + 10$
- $f_2(x, y) = 10x^2 + 3y$
- $f_3(n) = 2^n + 2^n$
- $f_4(m, n) = m \log n + n \log n$
- $f_5(x, y) = \max(x, y) + 3y + 10$
- $f_6(x, y) = \min(x, y)$

Exercise 3 (sample solution on page 16). Prove that $O(\max(x, y)) = O(x + y)$ for any non-negative integers x and y .

Hint: To show that $O(f) = O(g)$, you need to show that $f \in O(g)$ and $g \in O(f)$.

Exercise 4 (sample solution on page 16). Prove that $O(\log(n!)) = O(n \log n)$. (This is a challenge exercise.)

Hint: proving that $\log(n!) \in O(n \log n)$ is rather easy. Proving that $n \log n \in O(\log(n!))$ is much harder: try expanding $n \log$ using the laws of logarithms, until you can spot $\log(n!)$ among the terms you produce. Then, try bounding the remaining terms by $\log(n!)$ again.

Exercise 5 (sample solution on page 17). Consider the following array, which we are half-way through sorting using selection sort:

0	1	2	3	4	5	6	7	8
2	3	4	9	8	6	7	5	

Sorted
Not yet sorted

1. How many comparisons does it take until the next number is sorted? How many swaps are performed to get there?
2. Then, give the number of comparisons and swaps happening in the next iteration.
3. Finally, give the total number of comparisons and swaps selection sort needs to sort this array from start to finish (including the elements marked as “sorted”).

Exercise 6 (sample solution on page 18). In this exercise, you will implement another sorting called insertion sort. In insertion sort, we iterate over all elements in the input array. As we examine each element, we perform a series of swaps to move it to the place it belongs in the sorted left side of the array.

For example, consider the array $[2, 4, 5, 3]$ (in math notation):

- We first consider the element 2 at index 0. Since there is nothing to its left, we don't need to do anything to sort the array up to this index.
- Next we consider 4 at index 1. The element to its left, 2, is smaller. If we understand $[2]$ as a sorted array, then 4 needs to remain to its right. Therefore no swap is necessary.
- Next we consider 5 at index 2. The array $[2, 4]$ to its left is sorted. Again 5 is larger than 4, so we don't need to perform any swap to make the array $[2, 4, 5]$ sorted.

- Finally, we consider 3 at index 3. This element is smaller than 5 to its left. Therefore, we will need to perform at least one swap to place 3 in its sorted position. Thus, we swap 3 with 5, obtaining the array [2, 4, 3, 5], where 3 is larger than the element to its left, 2. Therefore we swap 3 with 4, obtaining [2, 3, 4, 5]. Now, 3 is larger than 2, and therefore no further swaps are needed.
- Since we have gone through all the elements of the input array, the array we obtained, [2, 3, 4] is the final sorted array.

As you implement selection sort, draw pictures to figure out what portion of the array is sorted at each point. Express your findings as loop invariants using functions in the `array_util` library.

What is the run time of this sorting algorithm?

Sample Solutions

Solution of exercise 1

- `function1(A, n)` has a worst-case asymptotic complexity of $O(n^2)$ because it goes through all n elements of array `A` in the inner loop and does this n times because of the outer loop. Therefore, `count` is incremented $n \times n$ times, resulting in an overall complexity of $O(n^2)$.
- `function2(A, n)` has a worst-time asymptotic complexity of $O(1)$ because both loops make k iterations (where k is set to a constant, 5). In particular, the number of iterations is not influenced by the value of the parameters `A` and `n`. Thus, Regardless of length of input array, `count` is incremented a constant number of times (25 to be precise), which yields $O(1)$ as the worst-case complexity of this function.

Solution of exercise 2

- $f_1(x) \in O(x^4)$
- $f_2(x, y) \in O(x^2 + y)$
- $f_3(n) \in O(2^n)$
- $f_4(m, n) \in O((m + n) \log n)$
- $f_5(x, y) \in O(\max(x, y))$ or $f_5(x, y) \in O(x + y)$ — see next exercise
- $f_6(x, y) \in O(\min(x, y))$

Solution of exercise 3

- Showing that $\max(x, y) \in O(x + y)$ is quite simple since $\max(x, y) \leq x + y$ for any x and $y \geq 0$. Following the definition of big-O, we set $n_0 = 0$ and $c = 1$.
- Showing the other direction, i.e., that $x + y \in O(\max(x, y))$, is almost as simple. The key observation here is that $x + y \leq 2 \max(x, y)$ for all x and $y \geq 0$. Formally, we would pick $n_0 = 0$ and $c = 2$.

In summary, $O(\max(x, y)) = O(x + y)$ because

$$\max(x, y) \leq x + y \leq 2 \max(x, y)$$

for all non-negative x and y .

Solution of exercise 4

- $\log(n!) \in O(n \log n)$:

$$\begin{aligned}
 \log(n!) &= \log(1 \times 2 \times \dots \times n) \\
 &= \log 1 + \log 2 + \dots + \log n \\
 &\leq \log n + \log n + \dots + \log n \\
 &= n \log n
 \end{aligned}$$

- $n \log n \in O(\log(n!))$:

$$\begin{aligned}
 n \log n &= \log(n) + \log(n) + \dots + \log(n) \\
 &= \log\left(\frac{n}{n}\right) + \log\left(\frac{n}{n-1}\right) + \dots + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{1}\right) \\
 &= \log\left(\frac{n}{n}\right) + \log\left((n-1)\frac{n}{n-1}\right) + \dots + \log\left(2\frac{n}{2}\right) + \log\left(1\frac{n}{1}\right) \\
 &= \log n + \log(n-1) + \dots + \log 2 + \log 1 + \\
 &\quad \log\left(\frac{n}{n}\right) + \log\left(\frac{n}{n-1}\right) + \dots + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{1}\right) \\
 &= \log(n!) + \\
 &\quad \log\left(\frac{n}{n}\right) + \log\left(\frac{n}{n-1}\right) + \dots + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{1}\right) \\
 &\leq \log(n!) + \log(n!) \\
 &= 2 \log(n!)
 \end{aligned}$$

Solution of exercise 5

1. It takes 4 comparisons and 1 swap to sort the next number. At first, the element 9 is picked as the current minimum. It is compared to 8, which now becomes the minimum. This process is repeated with 7, 6, and 5, which yields 5 as the overall minimum. Then 5 is swapped with 9.

Observe that 4 is one minus the length of the part of the array that is still to be sorted.

2. As we start the next iteration, the array is

0	1	2	3	4	5	6	7	8
2	3	4	5	8	6	7	9	

{ Sorted { Not yet sorted

We make 3 comparisons and one swap to sort the next element: 8 is first compared with 6, which becomes the current minimum; then 6 is then compared with 7, and again with 9. By then, 6 is retained as the minimum element, and it is swapped with 8.

Again, 3 is one minus the length of the part of the array that remains to be sorted.

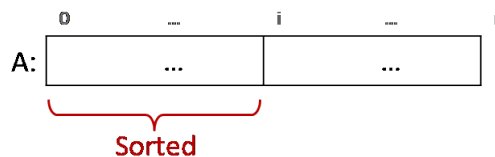
3. The total number of comparisons is $1 + 2 + 3 + \dots + 7 = 28$ and the number of swaps is 8 (including situations where selection sort swaps an element with itself).

In general, to sort an n -element array using selection sort, we will need to make

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

comparisons and n swaps.

Solution of exercise 6 As we start an iteration of the outer loop, say examining index i , the array segment to the left of i is sorted, while we don't know anything the rest of the array. This intuition is captured by the following picture:

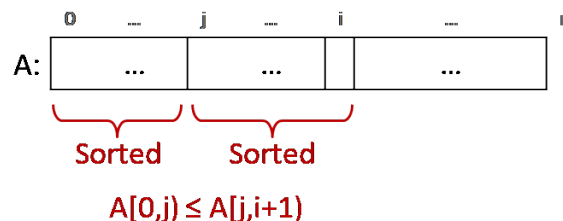


We can translate this insight using two loop invariants:

```
//@loop_invariant 0 <= i && i <= n;
//@loop_invariant is_sorted(A, 0, i);
```

The first describe the relation between the indices, and the second the fact that the left portion of the array is sorted.

The inner array moves the element picked by the outer loop leftward. Say at the beginning of an iteration, this element is at index j . Then the portion of the array to its left (from 0 to j excluded) is sorted (because it was in the outer loop), and so is the fragment between j and i included (because our element is smaller than anything in there, and these elements were sorted). This other picture captures this insight.



This gives rise to the following loop invariants:

```

//@loop_invariant 0 <= j && j <= i;
//@loop_invariant is_sorted(A, 0, j);
//@loop_invariant is_sorted(A, j, i+1);
//@loop_invariant le_segs(A, 0, j, A, j+1, i+1);

```

The code for insertion sort is as follows:

```

void inssort(int[] A, int n)
//@requires 0 <= n && n <= \length(A);
//@ensures is_sorted(A, 0, n);
{
    for (int i = 0; i < n; i++)
        //@loop_invariant 0 <= i && i <= n;
        //@loop_invariant is_sorted(A, 0, i);
        {
            int j = i;

            while (j > 0 && A[j-1] > A[j])
                //@loop_invariant 0 <= j && j <= i;
                //@loop_invariant is_sorted(A, 0, j);
                //@loop_invariant is_sorted(A, j, i+1);
                //@loop_invariant le_segs(A, 0, j, A, j+1, i+1);
                {
                    swap(A, j-1, j);
                    j--;
                }
        }
}

```

The inner loop of selection sort could swap the element examined by the outer loop with every value to its left (that's $i - 1$ swaps). In the worst case, this may happen for each element picked by the outer loop. The overall number of swaps is therefore

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

The worst-case asymptotic complexity is therefore $O(n^2)$.